Zustand centraliza el estado de la aplicación mediante un patrón de flujo de datos unidireccional, eliminando el prop drilling y garantizando que las animaciones de Framer Motion no sufran caídas de frames por re-renderizados innecesarios en componentes hermanos.

Para un MVP con arquitectura PWA, la estrategia ideal es consolidar la economía, el inventario y el álbum en un único store utilizando el patrón de Sincronización Optimista (Optimistic Updates)

Arquitectura del Store (El Árbol de Estado)

El estado se divide lógicamente en datos estáticos (catálogo) y datos mutables (economía e inventario del piloto

```
import { create } from 'zustand';

export const useGarageStore = create((set, get) => ({
  // --- ESTADO MUTABLE: Economía y Perfil ---
  racingCredits: 0,
  unopenedCrates: 0,

  // --- ESTADO MUTABLE: Colección ---
  pastedVehicles: [], // Vehículos ya estampados en el álbum
  inventory: [],      // Vehículos listos para pegar o intercambiar
  // Ej de objeto en array: { id: 89, name: "Fiat Uno 1.4s Confort 1997", category: "Clásicos Urbanos", count: 2 }

  // --- ACCIONES: Inyección y Gasto de Economía ---
  setInitialState: (data) => set({
    racingCredits: data.credits,
```

```
  unopenedCrates: data.crates,
  pastedVehicles: data.pasted,
  inventory: data.inventory
}),

addCreditsFromTrailer: (amount) => set((state) => ({
  racingCredits: state.racingCredits + amount
})),

buyCrate: (cost) => set((state) => ({
  racingCredits: state.racingCredits - cost,
  unopenedCrates: state.unopenedCrates + 1
})),

// --- ACCIONES: Core Loop (Unboxing y Pegado) ---
openCrate: (droppedVehicles) => set((state) => ({
  unopenedCrates: state.unopenedCrates - 1,
  // Agrega los 5 vehículos nuevos al inventario general
  inventory: [...state.inventory, ...droppedVehicles]
})),

// Sincronización Optimista: Actualiza la UI antes de que el servidor responda
pasteVehicle: async (vehicleId, apiCall) => {
  // 1. Clonar estado actual para un posible Rollback
  const previousInventory = [...get().inventory];
  const previousPasted = [...get().pastedVehicles];
```

```
    const vehicleToPaste = previousInventory.find(v => v.id === vehicleId);

    // 2. Actualización Optimista (La UI reacciona al instante, la carta se pega)
    set((state) => ({
      inventory: state.inventory.filter(v => v.id !== vehicleId),
      pastedVehicles: [...state.pastedVehicles, vehicleToPaste]
    }));

    try {
      // 3. Petición en segundo plano al backend (Python/SQLite)
      await apiCall(vehicleId);
    } catch (error) {
      // 4. Rollback en caso de fallo transaccional (ej. error 500 de SQLite)
      console.error("Transacción ACID fallida. Revirtiendo estado visual.");
      set({
        inventory: previousInventory,
        pastedVehicles: previousPasted
      });
    }
  }
}));
```

Consumo en los Componentes React

Al importar el store en las vistas, Zustand permite suscribirse únicamente a los fragmentos de estado necesarios. Esto es crucial para el rendimiento de la UI.

En la Barra de Navegación (Header):

```
import { useGarageStore } from '../store/useGarageStore';


export default function Header() {
  // El header solo se re-renderiza cuando cambian los créditos, no cuando se pega un auto.
  const racingCredits = useGarageStore((state) => state.racingCredits);


  return (
    <header className="flex justify-between items-center bg-zinc-900 text-white p-4">
      <h1>Escudería</h1>
      <div className="font-mono text-yellow-400">{racingCredits} CR</div>
    </header>
  );
}
```

//////////////////————————————————//////////////////

En el Álbum (GarageView.jsx):

```
import { useGarageStore } from '../store/useGarageStore';


export default function GarageView() {
  const pastedVehicles = useGarageStore((state) => state.pastedVehicles);
  const pasteVehicle = useGarageStore((state) => state.pasteVehicle);


  const handleDragEnd = (vehicleId) => {
    // Se ejecuta la acción asíncrona pasando la función fetch hacia la API
```

```
    pasteVehicle(vehicleId, async (id) => {
      const response = await fetch('/api/paste-vehicle', {
        method: 'POST',
        body: JSON.stringify({ vehicle_id: id })
      });
      if (!response.ok) throw new Error('Database transaction failed');
    });
  };

  // Renderizado del componente...
}
```

(((Justificación Técnica para el Jurado

La implementación de la Sincronización Optimista (pasteVehicle) es un punto fuerte para defender ante el jurado:

Experiencia de Usuario (UX): Los usuarios de sistemas móviles asumen interacciones de latencia cero. Si al soltar una carta en el álbum la interfaz se bloquea con un spinner de carga durante 300ms mientras SQLite procesa el bloqueo transaccional, el juego se percibe lento.

Integridad Garantizada: Al mutar la UI primero, el usuario obtiene su respuesta visual instantánea. Si por alguna razón la transacción en la base de datos falla o es rechazada por lógica de negocio, el bloque catch ejecuta el rollback devolviendo la carta al inventario y manteniendo el cliente y el servidor en perfecta sincronía.)))